



5. Llistes

RA6

MP0485 - Programació

Curs 2025-2026



1. Què és una llista?

Un **llista** és un tipus de dada complexa, que **permet guardar dades de qualsevol tipus**.

- Són mutables i **dinàmiques**.
- Permeten emmagatzemar **dades** de tipus **diferents**.

Exemple 1: només strings

Sonia	Joan	Pau	Aitor	Júlia	Blai	Ot	Ana	Laura	Pep	Àlex	Laia
-------	------	-----	-------	-------	------	----	-----	-------	-----	------	------

Exemple 2: strings i enters

Sonia	2	Pau	5.5	Júlia	7	Ot	4	Laura	5	Àlex	8.5
-------	---	-----	-----	-------	---	----	---	-------	---	------	-----



1. Què és una llista?

Les **l·listes** es poden fer servir per recrear **diferents estructures de dades**.

Vector o Array dinàmic

- Col·lecció ordenada d'elements accessible per índex.
- La seva mida pot créixer i reduir-se dinàmicament.
- En Python, les l·listes funcionen internament com a arrays dinàmics.

Matriu o Taula

- Estructura rectangular amb un nombre fix de files i columnes ($n \times m$).
- És habitualment **bidimensional (2D)**, tot i que poden existir matrius d'una fila, d'una columna o de més dimensions.

Cua o llista FIFO (queue)

- Llista on el primer en entrar és el primer en sortir (First In, First Out).
- En Python es pot implementar amb `append()` i `pop(0)`.

Pila o llista LIFO (stack)

- Llista on l'últim en entrar és el primer en sortir (Last In, First Out).
- En Python es pot implementar amb `append()` i `pop()`.



2. Posició (índex)

Per referir-nos a **un element** de la llista fem servir **l'índex**, és a dir, la seva posició dins la llista.

L'índex sempre comença al 0!

```
llista = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(f"Llista inicial: {llista}")
print(f"Element en la posició {1} {llista[1]}")
llista[9] = 20
print(f"Llista modificada: {llista}")
```

llista =

0	1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9	20



2. Posició (índex)

Per referir-nos a **un element** de la llista fem servir **l'índex**, és a dir, la seva posició dins la llista.

L'índex sempre comença al 0!

```
llista = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(f"Llista inicial: {llista}")
print(f"Element en la posició {1} {llista[1]}")
llista[9] = 20
print(f"Llista modificada: {llista}")
```

Llista inicial: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Element en la posició 1 2

Llista modificada: [1, 2, 3, 4, 5, 6, 7, 8, 9, 20]

llista =

0	1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9	20



3. IndexError

Què passa si intentem **accedir** a una posició que no existeix?

```
llista = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
print(llista[10])
```

0	1	2	3	4	5	6	7	8	9

Error

IndexError

Traceback (most recent call last)

Cell In[40], line 2

```
1 llista = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
----> 2 print(llista[10])
```

IndexError: list index out of range

**Cal controlar sempre
que els índexs estiguin
dins del rang!**



4. Longitud d'una llista

Per saber la longitud d'una llista, podem utilitzar la funció **len(llista)**, que retorna el número de columnes de la primera fila.

També ho podem fer amb un **comptador** per actualitzar la longitud amb menys cost, si estem afegint i eliminant elements constantment.

```
llista_1D = [1,2,3,4]
n= len(llista_1D)
print(f"Aquesta és una llista d'una dimensió amb
{n} elements: ")
print(llista_1D)
```

Aquesta és una llista d'una dimensió amb 4 elements:

```
[1, 2, 3, 4]
```

```
llista_1D = [1,2,3,4]
n= len(llista_1D)
print(f"Aquesta és una llista d'una dimensió amb
{n} elements: ")
print(llista_1D)
```

Aquesta és una llista de dues dimensions 3 * 4 amb 12 elements:

```
[[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]
```



5. Creació i visualització d'una llista

Les **l·listes** es poden crear **buides** o **amb valors inicials**.

```
l·lista = []  
l·lista = [1, 2, 3, 4]  
l·lista = [{"Paula", 2}, {"Joan", 5}]
```

- La l·lista es delimita amb parèntesis.
- Els elements van separats per comes.
- Es poden ficar l·listes dins d'una l·lista.

Es pot crear l'**estructura** "buida" multiplicant un valor (1D) o amb un for anidat (2D).

```
l·lista = [0]*5  
print(l·lista)
```

[0, 0, 0, 0, 0]

Es pot **visualitzar** una l·lista **recorrent-la amb un for**.

```
for valor in l·lista:  
    print(valor, end=" ")
```




6. Inserció d'elements

Es poden **afegir elements** amb **append(valor)**.

```
llista = []  
llista.append(5)  
llista.append(2)  
llista.append("Hola")  
print(llista)
```

[5, 2, 'Hola']

- Els elements afegits amb append s'insereixen al final.

També podem triar la posició (índex) on s'insereix un element fent servir **insert(index, valor)**

```
llista = [2,3,2]  
llista.insert(1,5)  
print(llista)
```

[2, 5, 3, 2]

- L'element s'afegirà a la posició triada i la resta es mouran una posició.
- Si indiquem una posició més gran de la longitud de la llista, s'afegirà igualment al final.



7. Eliminació d'elements

Per **eliminar** un element **per posició** cal fer servir la funció **pop(index)**.

```
llista = [2,3,4]
element = llista.pop(1)
print(f"Element eliminat: {element}")
print(f"Llista: {llista}")
```

```
Element eliminat: 3
Llista: [2, 4]
```

- La funció pop(i) ens permet també obtenir el valor de l'element eliminat.
- Si no especifiquem índex, la funció pop() elimina l'últim element de la llista.

Per **eliminar** un element **per valor**, podem fer servir la funció **remove(valor)**.

```
llista = [2,3,2]
llista.remove(2)
print(llista)
```

```
[3, 2]
```

- Només elimina la primera aparició del valor a la llista.
- No ens permet guardar el valor a una variable.



8. Ús amb bucles: sense llistes vs amb llistes

Vull fer un programa per **guardar les notes de 12 alumnes** de l'aula. Les **introduiré manualment**, totes seguides.

Sense llistes

```
nota_alumne1 = float(input("Introdueix la nota de l'alumne 1: "))
nota_alumne2 = float(input("Introdueix la nota de l'alumne 2: "))
nota_alumne3 = float(input("Introdueix la nota de l'alumne 3: "))
nota_alumne4 = float(input("Introdueix la nota de l'alumne 4: "))
nota_alumne5 = float(input("Introdueix la nota de l'alumne 5: "))
nota_alumne6 = float(input("Introdueix la nota de l'alumne 6: "))
nota_alumne7 = float(input("Introdueix la nota de l'alumne 7: "))
nota_alumne8 = float(input("Introdueix la nota de l'alumne 8: "))
nota_alumne9 = float(input("Introdueix la nota de l'alumne 9: "))
nota_alumne10 = float(input("Introdueix la nota de l'alumne 10: "))
nota_alumne11 = float(input("Introdueix la nota de l'alumne 11: "))
nota_alumne12 = float(input("Introdueix la nota de l'alumne 12: "))

print(f"{nota_alumne1}, {nota_alumne2}, ... , {nota_alumne12}")
```

Amb llistes

```
notes_examen = []
for i in range(12):
    nota = float(input(f"Introdueix la nota de
                        l'alumne {i + 1}: "))
    notes_examen.append(nota)

print("Notes dels alumnes:")
print(notes_examen)
```



8. Ús amb bucles: sense llistes vs amb llistes

Vull fer un programa per **guardar les notes de 12 alumnes** de l'aula. Les **introduiré manualment**, totes seguides.

Sense llistes

```
nota_alumne1 = float(input("Introdueix la nota de l'alumne 1: "))
nota_alumne2 = float(input("Introdueix la nota de l'alumne 2: "))
nota_alumne3 = float(input("Introdueix la nota de l'alumne 3: "))
nota_alumne4 = float(input("Introdueix la nota de l'alumne 4: "))
nota_alumne5 = float(input("Introdueix la nota de l'alumne 5: "))
nota_alumne6 = float(input("Introdueix la nota de l'alumne 6: "))
nota_alumne7 = float(input("Introdueix la nota de l'alumne 7: "))
nota_alumne8 = float(input("Introdueix la nota de l'alumne 8: "))
nota_alumne9 = float(input("Introdueix la nota de l'alumne 9: "))
nota_alumne10 = float(input("Introdueix la nota de l'alumne 10: "))
nota_alumne11 = float(input("Introdueix la nota de l'alumne 11: "))
nota_alumne12 = float(input("Introdueix la nota de l'alumne 12: "))

print(f"{nota_alumne1}, {nota_alumne2}, ... , {nota_alumne12}")
```

Amb llistes

```
notes_examen = []
for i in range(12):
    nota = float(input(f"Introdueix la nota de
                        l'alumne {i + 1}: "))
    notes_examen.append(nota)

print("Notes dels alumnes:")
print(notes_examen)
```

La posició a la llista és un menys que la posició a la llista d'alumnes. Haureu de vigilar aquesta diferència als programes.

Els programadors pensen en el 0 com a inici, però la resta no.



9. Avantatges: més flexibilitat

Vull fer un programa per **guardar les notes d'un nombre concret d'alumnes**.

Sense arrays

Es pot fer definint un valor màxim i amb l'ús de molts condicionals, molt poc eficient i gens pràctic.

Amb llistes

```
mida = int(input("Introdueix el nombre d'alumnes: "))
notes = []

for i in range(mida):
    nota = float(input(f"Introdueix la nota número {i + 1}: "))
    notes_examen.append(nota)
```



10. Altres exemples

Vull fer un programa per **guardar les notes de 12 alumnes** de l'aula. Vull poder **escollir** a quin alumne posar-li la nota.

```
mida = 12
notes = [None]*mida

alumne = int(input("Introdueix el número de llista de l'alumne a qui vols posar nota. (-1 per sortir): "))

while alumne != -1:
    nota = float(input(f"Introdueix la nota de l'alumne número {alumne}: "))
    notes[alumne] = nota
    alumne = int(input("Introdueix el número de llista de l'alumne a qui vols posar nota. (-1 per sortir): "))
```



10. Index out of bounds

Vull fer un programa per **guardar les notes de 12 alumnes** de l'aula. Vull poder **escollir** a quin alumne posar-li la nota.

```
mida = 12
notes = [None]*mida
```

Què passa si la persona introdueix un 12?

```
alumne = int(input("Introdueix el número de llista de l'alumne a qui vols posar nota. (-1 per sortir): "))

while alumne != -1:
    nota = float(input(f"Introdueix la nota de l'alumne número {alumne}: "))
    notes[alumne] = nota
    alumne = int(input("Introdueix el número de llista de l'alumne a qui vols posar nota. (-1 per sortir): "))
```



10. Comprovació de la posició

Com hem vist, si **intenteu accedir** a una posició que **surt** de la llargada de l'array, us donarà un **error**.

Heu de **comprovar** sempre, en demanar un índex per teclat, **que aquest estigui dins dels límits (0-(N-1))**

```
mida=12
notes = [None]*mida
alumne = int(input("Introdueix el número de llista de l'alumne a qui vols posar nota. (-1 per sortir): "))

while alumne != -1:
    while not (alumne >=0 and alumne < mida):
        print("La posició introduïda no és vàlida!")
        alumne = int(input("Introdueix el número de llista de l'alumne a qui vols posar nota. (-1 per sortir): "))

    nota = float(input(f"Introdueix la nota de l'alumne número {alumne}: "))
    notes[alumne] = nota
    alumne = int(input("Introdueix el número de llista de l'alumne a qui vols posar nota. (-1 per sortir): "))
```




10. Referència a la mida de la llista

Si heu de fer referència moltes vegades a la **mida** de l'array, **definiu-vos una variable**:

```
mida=12
notes = [None]*mida
alumne = int(input("Introdueix el número de llista de l'alumne a qui vols posar nota. (-1 per sortir): "))

while alumne != -1:
    while not (alumne >=0 and alumne < mida):
        print("La posició introduïda no és vàlida!")
        alumne = int(input("Introdueix el número de llista de l'alumne a qui vols posar nota. (-1 per sortir): "))

    nota = float(input(f"Introdueix la nota de l'alumne número {alumne}: "))
    notes[alumne] = nota
    alumne = int(input("Introdueix el número de llista de l'alumne a qui vols posar nota. (-1 per sortir): "))
```



11. Slicing

El **slicing** permet **extreure un fragment** d'un string o llista mitjançant la seva posició.

- El més habitual és extreure valors consecutius.
- **El valor inicial sempre s'inclou, però el final no.**

```
llista = [0, 1, 2, 3, 4, 5]
```

```
#Visualitzar el segon element de la llista  
print(llista [1])
```

1

```
#Visualitzar els dos primers elements de la llista  
print(llista [:2])
```

[0, 1]

```
#Visualitzar des de la posició dos fins a l'últim  
print(llista [2:])
```

[2, 3, 4, 5]

```
#Visualitzar de la posició 2 a la 4  
print(llista [2:5])
```

[2, 3, 4]



11. Vistes i còpies

Les **vistes** permeten definir una nova variable per **visualitzar la llista** (sencera o una part), però si es modifiquen, també **es modifica llista original**.

```
l = [0,0,0,0,0]
l2 = l[0:2]

print(f"Llista: {l}")
print(f"Vista: {l2}")

l2[0] = 1

print(f"Llista: {l}")
print(f"Vista: {l2}")
```

```
Llista: [0, 0, 0, 0, 0]
Vista: [0, 0]
Llista: [0, 0, 0, 0, 0]
Vista: [1, 0]
```

Si necessitem fer una **còpia** d'una llista (total o parcial), que no afecti a l'original, **s'ha de copiar cridant al mètode .copy()**.

```
l = [0,0,0,0,0]
l2 = l[0:2].copy()

print(f"Llista: {l}")
print(f"Còpia: {l2}")

l2[0] = 1

print(f"Llista: {l}")
print(f"Còpia: {l2}")
```

```
Llista: [0, 0, 0, 0, 0]
Còpia: [0, 0]
Llista: [0, 0, 0, 0, 0]
Còpia: [1, 0]
```



12. Exemples per practicar

- Fes un programa que generi 20 nombres aleatoris del 1 al 100 i els guardi en una llista. Després imprimeix els nombres de la llista.
- Modifica el programa anterior perquè després de crear la llista, imprimeixi només els nombres parells.
- Fes un programa que pregunti a cada persona de la classe quin és el resultat d'una suma amb nombres aleatoris del 1 al 100. Heu de tenir una llista amb els noms dels companys de classe guardats. Si la persona falla, haurà de tornar a intentar-ho. Només passem de persona quan encerta.

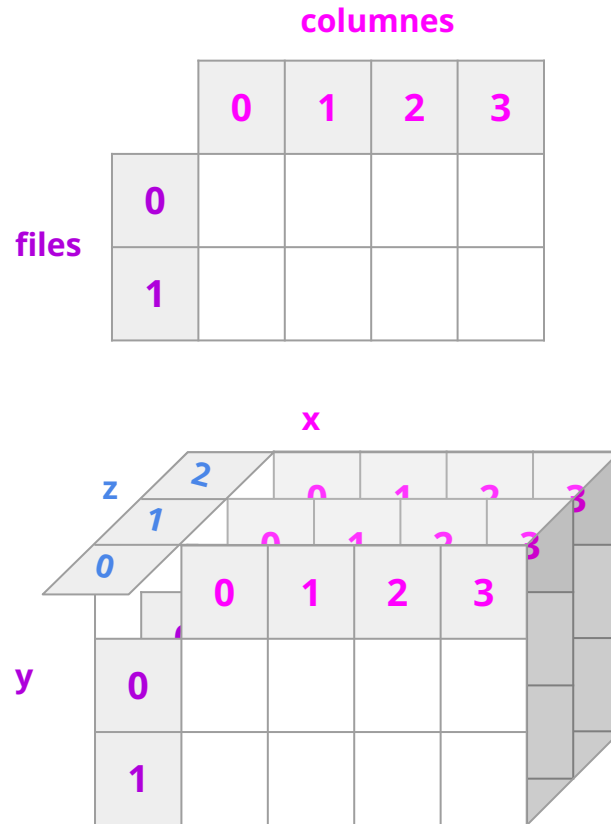
```
alumnes = ["Oleguer", "Felix", "Eric", "Daniel", "Àlex", "Izan", "Pol", "Adrià", "Joaquím", "Hicham", "Xinhao"]
```



13. Llistes multidimensionals

Les llistes poden tenir tantes **dimensions** com vulguem. Això permet indexar més dades.

- **Accés:** igual que fèiem abans, haurem d'indicar la posició on volem accedir (l'índex) entre claudàtors, però per a cadascuna de les dimensions i separades amb una coma. Per això és important saber l'ordre (què són les files i què les columnes).
- **Utilitat:** les llistes de dues dimensions poden ser especialment útils per guardar dades del mateix tipus per a diversos usuaris (equivalent a taula en base de dades) o bé per representar dades espacials (mapes, imatges...).





14. Llistes com a matrius

Les **matrius** són de **mida fixa** i habitualment de dues dimensions. És a dir, tenen un nombre inicial de files i columnes que no es modifica.

		columnes			
		0	1	2	3
files	0	1.	2.	0.	0.
	1	0.	0.	0.	8.

```
files = 2
cols = 4
m = []
for i in range(files):
    fila = [0]*cols
    m.append(fila)

m[0][0]=1
m[0][1]=2
m[1][3]=8

for fila in(m):
    for col in(fila):
        print(col, end=" ")
    print()

print(m[1][0]) #Imprimirà 0 per pantalla
```



14. Llistes com a matrius

Les **matrius** són de **mida fixa** i habitualment de dues dimensions. És a dir, tenen un nombre inicial de files i columnes que no es modifica.

		columnes			
		0	1	2	3
files	0	1.	2.	0.	0.
	1	0.	0.	0.	8.

Una vegada creada la matriu, no podem fer appends o desmuntarem l'estructura de la taula!

```
files = 2
cols = 4
m = []
for i in range(files):
    fila = [0]*cols
    m.append(fila)

m[0][0]=1
m[0][1]=2
m[1][3]=8

for fila in(m):
    for col in(fila):
        print(col, end=" ")
    print()

print(m[1][0]) #Imprimirà 0 per pantalla
```



14. Matriu 2D - Exemple

Vull fer un programa per **gestionar** les vendes mensuals d'una botiga, de les 5 categories de productes diferents que tenen.

```
def crear_matriu(files, cols):  
    m = []  
    for i in range(files):  
        fila = [0]*cols  
        m.append(fila)  
    return m  
  
def imprimir_matriu(m):  
    for fila in m:  
        for col in fila:  
            print(col, end=" ")  
        print()
```

```
mesos = 12  
prods = 5  
vendes_mensuals = crear_matriu(mesos,prods)  
imprimir_matriu(vendes_mensuals)
```

files =
mesos

columnes =
productes

	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0
5	0	0	0	0	0
...	0	0	0	0	0
11	0	0	0	0	0



14. Matriu 2D - Exemple

Vull fer un programa per **gestionar** les vendes mensuals d'una botiga, de les 5 categories de productes diferents que tenen.

```
#Introduir vendes d'un mes, categoria concreta
mes = int(input("A quin mes vols introduir les vendes (1-12)?"))-1 #5
cat = int(input("A quina categoria (1-5)?"))-1 #5
vendes = int(input("Indica la quantitat:")) #50
vendes_mensuals[mes][cat] = vendes

#Visualitzar vendes d'un mes, categoria concreta
mes = int(input("De quin mes vols consultar les vendes (1-12)?"))-1 #2
cat = int(input("De quina categoria (1-5)?"))-1 #3
vendes = vendes_mensuals[mes][cat] #Mostrarà 0
print(f"Les vendes del mes {mes+1} a la categoria {cat+1} van ser: {vendes}.")
```

files =
mesos

columnes =
productes

	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0
5	0	0	0	0	50
...	0	0	0	0	0
11	0	0	0	0	0



14. Matriu 2D - Exemple

```
for mes in range(mesos):
    for cat in range(productes):
        vendes_mensuals[mes][cat] = mes+1    #Les dades introduïdes faciliten la
        comprovació del funcionament del codi

#Visualitzar vendes d'un mes, per totes les categories
mes = int(input("De quin mes vols visualitzar les vendes (1-12)?"))-1 #4
for v in vendes_mensuals[mes]:
    print(v, end=" ")    # 4 4 4 4 4

print()

#Visualitzar vendes d'una categoria, per tots els mesos
cat = int(input("De quina categoria vols visualitzar les vendes (1-5)?"))-1 #3
for fila in vendes_mensuals:
    print(fila[cat], end=" ") # 1 2 3 4 5 6 7 8 9 10 11 12

print()
```

	0	1	2	3	4
0	1	1	1	1	1
1	2	2	2	2	2
2	3	3	3	3	3
3	4	4	4	4	4
4	5	5	5	5	5
5	6	6	6	6	6
...	...	...	...	...	...
11	12	12	12	12	12



15. Webgrafia

- <https://cupi2-ip.github.io/IPBook/nivel3/seccion3-4.html>

□ **CC BY 4.0**

Contingut elaborat per Cristina Reina León.

<https://creativecommons.org/licenses/by/4.0/>